

Terraform - Cheat Sheet

Examples assume Terraform 1.10 or newer, with the AWS or Google provider. Version gates are called out in-line. For the command table, jump to [Quick reference](#).

Contents

Getting started

1. [The workflow](#)
2. [HCL essentials](#)

Declaring infrastructure

3. [Providers](#)
4. [Resources](#)
5. [Data sources](#)

Inputs, outputs and state

6. [Variables](#)
7. [Outputs](#)
8. [State](#)

Reuse and environments

9. [Modules](#)
10. [Workspaces](#)

Resource behaviour

11. [Lifecycle and meta-arguments](#)
12. [Dynamic blocks](#)
13. [Functions](#)

Operations and patterns

14. [Targeted operations and replacement](#)
15. [Importing existing resources](#)
16. [Common patterns](#)
17. [Terragrunt](#)

Reference

18. [Defaults to follow](#)

19. [Common gotchas](#)

20. [Quick reference](#)

Terraform is HashiCorp's infrastructure-as-code tool. We write declarative `.tf` files describing what cloud resources we want, run `terraform apply`, and Terraform figures out how to make reality match. It tracks what it created in a state file and only changes the resources that drifted.

It works across AWS, GCP, Azure, Kubernetes, Datadog, GitHub and thousands more providers, with the same syntax everywhere. Against a cloud-native tool like CloudFormation, we trade a little extra setup for that portability.

1. The workflow

Four commands cover almost all daily use.

```
terraform init      # download providers, initialize backend
terraform plan      # show what would change
terraform apply     # make it happen
terraform destroy   # tear it all down
```

`plan` is read-only. Always run it before `apply` so we know exactly what is going to change.

```
terraform fmt      # format files
terraform validate  # check types and references, no API calls
terraform show     # current state
terraform state list # list resources in state
```

`validate` needs an initialised directory, because it loads provider schemas to check resource arguments. It reaches no cloud API, so it is safe in CI on every commit.

2. HCL essentials

HCL (HashiCorp Configuration Language) is the syntax. It is declarative, with some imperative-looking constructs.

```

# A block: resource type + label + body
resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.micro"

  tags = {
    Name = "web-server"
    Env  = "prod"
  }
}

# Variable
variable "region" {
  type    = string
  default = "us-east-1"
}

# Reference: <resource_type>.<name>.<attribute>
output "web_ip" {
  value = aws_instance.web.public_ip
}

```

Types: `string`, `number`, `bool`, `list(T)`, `set(T)`, `map(T)`, `object({})`, `tuple([])`, `any`.

Interpolation:

```
name = "web-${var.environment}-${count.index}"
```

3. Providers

Providers are plugins that talk to cloud APIs. Declare them at the top:

```

terraform {
  required_version = ">= 1.10"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

provider "google" {
  project = "my-project"
  region  = "us-central1"
}

```

Pin versions. `~> 5.0` means "5.x but not 6.0". Terraform also writes `.terraform.lock.hcl`, recording the exact provider builds it chose. Commit it. Without it, a fresh `init` can pick up a newer patch and change a plan nobody edited.

Multiple instances of the same provider via aliases:

```

provider "aws" {
  region = "us-east-1"
  alias  = "east"
}

provider "aws" {
  region = "eu-west-1"
  alias  = "europe"
}

resource "aws_s3_bucket" "logs_eu" {
  provider = aws.europe
  bucket   = "my-logs-eu"
}

```

4. Resources

A resource is something Terraform creates and manages.

```

resource "aws_s3_bucket" "data" {
  bucket = "my-app-data"

  tags = {
    Owner = "team-platform"
  }
}

resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id

  versioning_configuration {
    status = "Enabled"
  }
}

```

The first label is the resource type (`aws_s3_bucket`). The second is the local name (`data`). Other resources reference this one as `aws_s3_bucket.data.<attribute>` .

5. Data sources

Read-only queries to look up existing resources or external info.

```

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
}

```

Useful for looking up account IDs, VPC IDs, AMIs, existing IAM roles, secrets in a vault.

A data source is read fresh on every plan. `most_recent = true` therefore resolves to a new AMI the moment Canonical publishes one. Because `ami` forces replacement, the next apply rebuilds the server. Pin the lookup when a stable answer matters more than a current one.

6. Variables

Inputs are declared with `variable` blocks and can be supplied from five places.

Variable declarations

```
variable "environment" {
  type      = string
  description = "Deployment environment"
  default    = "dev"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}

variable "db_password" {
  type      = string
  sensitive = true
}

variable "tags" {
  type      = map(string)
  default   = {}
}
```

Values

Terraform resolves inputs in this order, lowest precedence first. Later wins:

1. Environment variables: `TF_VAR_environment=prod`.
2. `terraform.tfvars`.
3. `terraform.tfvars.json`.
4. `*.auto.tfvars` and `*.auto.tfvars.json`, in lexical order.
5. Command-line `-var` and `-var-file`, in the order given.

The command line wins, which is what makes it usable for a one-off override. Auto-loaded files sit near the bottom, so a committed `terraform.tfvars` cannot quietly beat a flag set in CI.

`sensitive = true` keeps the value out of plan and apply output. It stays in state in the clear.

7. Outputs

Surface values that other tools or modules need.

```
output "vpc_id" {
  value      = aws_vpc.main.id
  description = "Main VPC ID"
}

output "db_password" {
  value      = aws_db_instance.main.password
  sensitive  = true
}
```

After `terraform apply`, outputs print and can be read with `terraform output vpc_id`.

8. State

The state file (`terraform.tfstate`) is Terraform's map between config and the real world. It tracks resource IDs, attributes, dependencies.

Why state matters

- Without state, Terraform cannot tell what already exists.
- State holds sensitive values (DB passwords, keys). Treat it like a secret.
- Two engineers running Terraform against the same target need shared state or they will fight.

Remote state

Local state files do not share. Production should always use remote state.

```
terraform {
  backend "s3" {
    bucket      = "my-tf-state"
    key         = "prod/network.tfstate"
    region      = "us-east-1"
    encrypt     = true
    use_lockfile = true
  }
}
```

The Google equivalent:

```
terraform {
  backend "gcs" {
    bucket = "my-tf-state"
    prefix = "prod/network"
  }
}
```

Backends:

- `s3` (AWS)
- `gcs` (Google Cloud Storage)
- `azurerm` (Azure)
- `remote` (Terraform Cloud / Enterprise)
- `http` (custom)

State locking

Two people applying at the same time can corrupt state. Locking blocks the second one.

- `s3` holds a lock file in the bucket itself with `use_lockfile = true`, from Terraform 1.10. Older setups pass a `dynamodb_table` instead.
- `gcs` locks natively, with nothing to configure.
- Terraform Cloud locks automatically.

Force-unlock only when we are certain no apply is still running:

```
terraform force-unlock <LOCK_ID>
```

9. Modules

A module is a reusable bundle of resources.

Module structure

```
modules/vpc/
  main.tf      # resources
  variables.tf # inputs
  outputs.tf   # outputs
  versions.tf  # provider requirements
```


Calling a module

```
module "vpc" {  
  source = "../modules/vpc"  
  
  name      = "prod-vpc"  
  cidr_block = "10.0.0.0/16"  
  az_count  = 3  
}  
  
output "vpc_id" {  
  value = module.vpc.vpc_id  
}
```

Module sources

```
# Local  
source = "../modules/vpc"  
  
# Git  
source = "git::https://github.com/example/modules.git//vpc?ref=v1.2.0"  
  
# Terraform Registry  
source  = "terraform-aws-modules/vpc/aws"  
version = "5.0.0"  
  
# S3  
source = "s3::https://s3.amazonaws.com/my-bucket/vpc.zip"
```

Pin versions when sourcing from a registry or git. An unpinned module can change between two applies that touched none of our own code.

10. Workspaces

Multiple states for one configuration. Each workspace keeps its own state file in the same backend. Terraform exposes the current name as `terraform.workspace`.

```
terraform workspace list  
terraform workspace new staging  
terraform workspace select prod
```

```
resource "aws_s3_bucket" "data" {  
  bucket = "myapp-data-${terraform.workspace}"  
}
```

Workspaces share one backend, one credential set and one configuration. They cannot express an environment that differs structurally, and moving from staging to prod is a single `select`. That is why most teams isolate environments by directory instead, as in [Common patterns](#). Workspaces suit variations that are not environments: a throwaway copy of a stack, or a per-developer sandbox.

11. Lifecycle and meta-arguments

Every resource accepts these special arguments.

count and for_each

```
# count: make N copies
resource "aws_instance" "web" {
  count      = 3
  ami       = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  tags = { Name = "web-${count.index}" }
}

# for_each: one per key in a map or set
resource "aws_iam_user" "developers" {
  for_each = toset(["alice", "bob", "charlie"])

  name = each.key
}
```

`for_each` is safer than `count`, because state addresses key off the map key rather than a position. Remove the first element of a `count` list and every later index shifts down. Terraform then destroys and recreates all of them to close the gap. For a list already in production, `moved` blocks re-point state without touching the resources.

depends_on

Explicit dependency when Terraform cannot infer one from a reference.

```
resource "aws_iam_role_policy" "lambda" {
  # ...
  depends_on = [aws_iam_role.lambda]
}
```

References build the graph on their own almost always. The gap opens where the real dependency is not an attribute we read. IAM roles and policies are the usual case. Terraform creates both in parallel, and the policy fails because the role is not live yet.

lifecycle block

```
resource "aws_s3_bucket" "important" {  
  bucket = "critical-data"  
  
  lifecycle {  
    prevent_destroy      = true  
    create_before_destroy = true  
    ignore_changes       = [tags]  
  }  
}
```

- `prevent_destroy`: refuses to delete this resource. Plan fails if it would.
- `create_before_destroy`: when replacing, create the new one first. Useful for downtime-sensitive resources.
- `ignore_changes`: stops Terraform reacting to drift on these attributes. Common for tags another tool owns, and the cost is that genuine drift there goes unreported too.

provider

Points a resource at a non-default provider alias.

12. Dynamic blocks

Generate nested blocks programmatically.

```
resource "aws_security_group" "web" {  
  name = "web"  
  
  dynamic "ingress" {  
    for_each = var.allowed_ports  
  
    content {  
      from_port = ingress.value  
      to_port   = ingress.value  
      protocol  = "tcp"  
      cidr_blocks = ["0.0.0.0/0"]  
    }  
  }  
}
```

If `var.allowed_ports = [80, 443]`, this produces two ingress blocks. Useful when the block count depends on input.

13. Functions

HCL has built-in functions for strings, collections, encoding and the filesystem.

```
upper("hello")           # -> "HELLO"
length([1, 2, 3])        # -> 3
concat([1, 2], [3, 4])    # -> [1, 2, 3, 4]
merge({a = 1}, {b = 2})   # -> {a = 1, b = 2}
lookup(map, "key", "default")
jsonencode({foo = "bar"}) # -> "{\"foo\":\"bar\"}"
file("${path.module}/script.sh")
templatefile("config.tpl", {name = "x"})
formatdate("YYYY-MM-DD", timestamp())
```

The language has no user-defined functions. Providers can ship their own from Terraform 1.8, reachable as `provider::::<function>`. Past those, pre-compute in a script and pass the result in as a variable.

14. Targeted operations and replacement

-target

Limit a plan or apply to specific resources.

```
terraform plan -target=aws_instance.web
terraform apply -target=module.vpc
```

Use sparingly. `-target` narrows the graph to that resource and its dependencies, leaving everything else unreconciled. State can settle into a shape no full apply would produce.

-replace

Mark a resource for replacement on the next apply.

```
terraform apply -replace=aws_instance.web
```

This replaces the older `terraform taint` workflow. Useful when a resource is in a bad state and needs recreating.

15. Importing existing resources

If a resource exists outside Terraform, import it into state.

```
import {
  to = aws_s3_bucket.existing
  id = "my-existing-bucket"
}

resource "aws_s3_bucket" "existing" {
  bucket = "my-existing-bucket"
}
```

The `import` block arrives in Terraform 1.5. It is the preferred route, because it runs through plan and can be reviewed. The CLI form still works: `terraform import aws_s3_bucket.existing my-existing-bucket`.

After import, run `terraform plan` to verify the config matches the real resource. Edit the config until plan shows no changes.

16. Common patterns

Multi-environment with directories

```
environments/
dev/
  main.tf
  backend.tf
  terraform.tfvars
staging/
prod/
modules/
vpc/
rds/
eks/
```

Each environment has its own state and inputs, and every one of them reuses the same modules.

Layered state (per concern)

Split state by lifecycle:

- Network (VPC, subnets): rarely changes.
- Data layer (RDS, S3): changes occasionally.
- Compute layer (EKS, Lambda, ECS): changes often.

Each layer has its own state. The compute layer reads from network state via the `terraform_remote_state` data source.

```

data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "my-tf-state"
    key    = "prod/network.tfstate"
    region = "us-east-1"
  }
}

resource "aws_instance" "web" {
  subnet_id = data.terraform_remote_state.network.outputs.private_subnet_ids[0]
}

```

17. Terragrunt

A thin wrapper around Terraform. It removes repeated config, sets up remote state, and orders applies by dependency between modules.

```

# terragrunt.hcl
include "root" {
  path = find_in_parent_folders()
}

terraform {
  source = "../../modules/vpc"
}

inputs = {
  cidr_block = "10.0.0.0/16"
  name      = "prod-vpc"
}

```

Popular for managing many small Terraform modules across environments. Skip it if Terraform alone is enough.

18. Defaults to follow

- Encrypt state at rest. Sensitive values land in state in the clear.
- Keep secrets out of `.tf` and `.tfvars` files in git. Use a secret manager or environment variables.
- Use modules for anything built more than once.
- Run `terraform fmt` in CI. Inconsistent formatting causes pointless diffs.
- Tag everything with `Environment`, `Team`, `Project`. Cloud console filters depend on it.
- Reach for `terraform_remote_state` sparingly. Every cross-layer read couples two states that then have to move together.

- Review every `prevent_destroy` removal. That flag exists for a reason.

19. Common gotchas

- State drift: someone edited a resource in the console, and the next plan proposes changes nobody asked for. Either pull the change into config with `import` or a rewrite, or put the resource back.
- Dependency cycle: two resources referencing each other directly. Refactor so one side reads from the other.
- Accidental replacement: a change to a `forces-new-resource` attribute, like an instance AMI, becomes a `destroy` and `create`. Read the plan output, not the summary.
- Stuck state lock: a previous apply crashed without releasing it. Wait for the TTL, or `force-unlock` once we are certain nothing is running.
- `terraform destroy` is too easy, and it deletes everything in the state. Scope it with `-target`, and remove lifecycle protections only when there is no other way.
- Provider auth surprises: a workspace or profile switch can mean different credentials. Verify with `aws sts get-caller-identity` or `gcloud config list` before applying to prod.

20. Quick reference

Command	What it does
<code>terraform init</code>	Download providers, initialize backend
<code>terraform plan</code>	Show what would change
<code>terraform apply</code>	Make changes
<code>terraform destroy</code>	Delete everything in state
<code>terraform fmt</code>	Format files
<code>terraform validate</code>	Check types and references locally (needs <code>init</code>)
<code>terraform show</code>	Inspect state
<code>terraform state list</code>	List resources in state
<code>terraform state mv</code>	Rename a resource in state
<code>terraform state rm</code>	Remove from state (without destroying)
<code>terraform import</code>	Bring an existing resource into state
<code>terraform output</code>	Show output values
<code>terraform workspace</code>	Manage workspaces
<code>terraform plan -target</code>	Plan one resource only

Command	What it does
<code>terraform apply -replace</code>	Force replace a resource

Concept	Key fact
Provider	Plugin for a cloud or service
Resource	A thing Terraform creates and manages
Data source	Read-only lookup, re-read on every plan
Variable	Input to a configuration
Output	Surface a value to callers
State	What Terraform thinks exists
Backend	Where state is stored
Lock	Prevents concurrent apply
Module	Reusable bundle
Workspace	Multiple states for the same code
<code>lifecycle</code>	Per-resource creation rules
<code>for_each</code>	One resource per key in a map or set
<code>dynamic</code>	Programmatic nested blocks